Contents

1. Objective . . . 3

2. Reference Tools . . . 3

3. Approach . . . 3

4. Architecture . . . 3

4.1 Project files . . . 4

4.2 Architecture . . . 4

4.3 Scaling and update configuration . . . 5

4.4 Running the project . . . 5

4.5 Measurement method . . . 5

5. Source Code . . . 6

5.1 app.py — versioned test service . . . 6

5.2 Dockerfile — image definition (VERSION / BROKEN build arguments) . . . 6

5.3 kind-config-task14.yaml — Kind cluster definition . . . 7

5.4 00-namespace.yaml — Namespace . . . 7

5.5 01-deployment.yaml — Deployment (rolling-update strategy) . . . 7

5.6 02-service.yaml — NodePort Service . . . 8

5.7 hpa.yaml — HorizontalPodAutoscaler . . . 8

5.8 deploy_scaling.py — experiment automation (lines 1–62) . . . 9

5.8 deploy_scaling.py — experiment automation (lines 63–124) . . . 10

5.8 deploy_scaling.py — experiment automation (lines 125–186) . . . 11

5.8 deploy_scaling.py — experiment automation (lines 187–248) . . . 12

5.8 deploy_scaling.py — experiment automation (lines 249–310) . . . 13

5.8 deploy_scaling.py — experiment automation (lines 311–339) . . . 14

6. Screenshots and Command Output . . . 15

6.1 Images and cluster . . . 15

6.2 Initial deployment . . . 17

6.3 Scaling replicas . . . 18

6.4 Rolling update . . . 19

6.5 Faulty release and rollback . . . 20

6.6 Resource utilisation and autoscaling . . . 21

6.7 Monitoring deployment status . . . 23

7. Results . . . 25

7.1 Scaling replicas . . . 25

7.2 Rolling update 1.0 → 2.0 . . . 25

7.3 Faulty release and rollback . . . 26

7.4 Resource utilisation and autoscaling . . . 27

7.5 Evaluation criteria . . . 28

8. Observations . . . 28

9. Known Limitations . . . 29

10. Conclusion . . . 29

1. Objective

To implement scalability and deployment-update strategies with Kubernetes: scale the replicas of an application up and down, roll out a new version without interrupting service, monitor the deployment while it changes, validate that the application stays available, and analyse the resource utilisation that drives automatic scaling.

2. Reference Tools

Kubernetes (on a three-node **Kind** cluster) provides the Deployment controller, ReplicaSets, the rolling-update strategy and the **HorizontalPodAutoscaler** (HPA). **metrics-server** supplies the CPU and memory metrics used by `kubectl top` and by the HPA. The workload is a small Flask service, `scale-app`, built three times from one Dockerfile: version 1.0 (initial), 2.0 (update) and 3.0 (a faulty release whose readiness endpoint always fails). A Python script drives every experiment and, in parallel, measures availability with a probe that sends ten requests per second through the Service.

3. Approach

1. **Scale application replicas** — manual scaling 3 → 6 → 2 → 3 with `kubectl scale`, timing how long all pods need to become Ready (Figure 5); automatic scaling with an HPA under CPU load (Figure 11).
2. **Perform rolling updates** — update the image from 1.0 to 2.0 with maxSurge 1 / maxUnavailable 0, then try a faulty 3.0 release and roll it back (Figure 6, Figure 8, Figure 9).
3. **Monitor deployment status** — rollout status and history, ReplicaSets, pod timelines sampled every 1.5 s, describe output and events (Figure 6, Figure 7, Figure 14).
4. **Validate application availability** — a continuous probe through the NodePort Service records every response (status, version, pod, latency) during the update, the faulty release and the autoscaling (Figure 7).
5. **Analyse resource utilisation** — `kubectl top` for pods and nodes before and during load, and the HPA's CPU-utilisation / replica timeline (Figure 10, Figure 11).

Everything is automated by `deploy_scaling.py`; commands and outputs are stored in `evidence/run_log.json` and all measurements in `evidence/results.json`, from which this report is generated.

4. Architecture

One Deployment (web) behind one NodePort Service. The Deployment owns one ReplicaSet per version; the HPA adjusts its replica count from metrics-server data; an external probe and load generator talk to the application through the Service.

4.1 Project files

File	Role
`app14/app.py`	Versioned Flask service: /, /version, /health (liveness), /ready (readiness; always fails in the faulty release) and /work?ms=N (burns CPU).
`app14/Dockerfile`	One Dockerfile, three images via build arguments VERSION and BROKEN; dependencies are installed before the arguments so layers are cached.
`kind-config-task14.yaml`	Cluster with one control-plane and two workers; host port 8100 mapped to NodePort 30100.
`k8s14/*.yaml`	Namespace scale-demo, Deployment web (3 replicas, RollingUpdate, probes, requests/limits) and NodePort Service web.
`k8s14/optional/hpa.yaml`	HorizontalPodAutoscaler: 3–6 pods, 50 % CPU of the request, quick scale-up and a 30 s scale-down window.
`deploy_scaling.py`	Automation, load generation, availability probe and measurement of all experiments.
`Task14_Scaling_RollingUpdates.ipynb`	Notebook with the same commands (alternative to the .py script).

4.2 Architecture

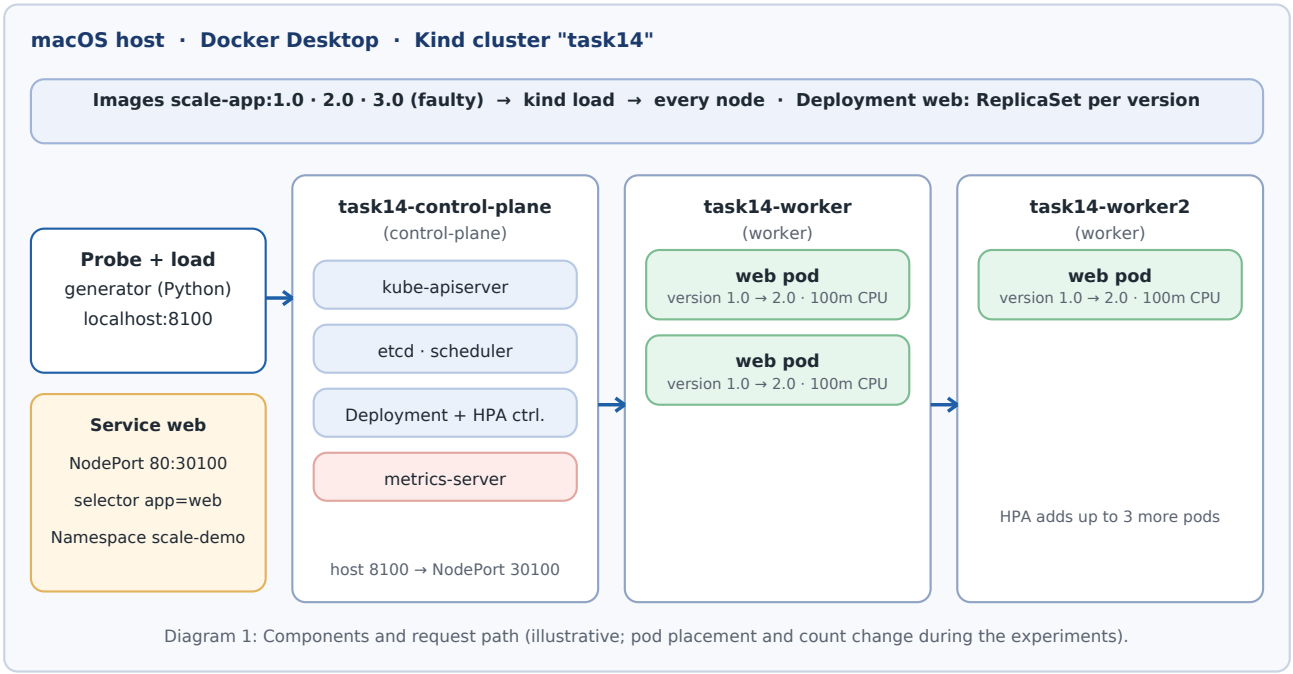


Diagram 1: Components and request path (illustrative; pod placement and count change during the experiments).

Requests enter through the host port mapping and the NodePort Service and are spread over the **ready** pods; pods that are not ready (new pods during an update, or a faulty release) receive no traffic. The HPA reads CPU usage from metrics-server and changes the Deployment's replica count; the Deployment controller then adds or removes pods.

4.3 Scaling and update configuration

Setting	Value	Purpose
replicas	3 (HPA range 3–6)	Baseline capacity; the autoscaler may add up to three pods.
strategy	RollingUpdate · maxSurge 1 · maxUnavailable 0	One extra pod is started and must become ready before an old pod is removed, so capacity never drops.
readinessProbe	GET /ready every 2 s	A new pod receives traffic only when its release is healthy; the faulty release never passes.
livenessProbe	GET /health every 10 s	Restarts a hung process without being affected by readiness.
progressDeadlineSeconds	60	A stalled rollout is reported as failed after one minute.
resources	requests 100m / 64Mi · limits 200m / 128Mi	Requests are the basis of the HPA's utilisation percentage; limits cap one pod at 200 % of its request.
revisionHistoryLimit	5	Old ReplicaSets are kept so that rollback is possible.
HPA	target 50 % CPU · scale-up +3 pods/15 s · scale-down window 30 s	Fast reaction to load and a shortened (default 300 s) cool-down so the demo finishes.

4.4 Running the project

1. `python deploy_scaling.py # about 15-20 minutes`
2. `python build_report.py --name "Your Name" --id "Your ID"`

Prerequisites: Docker Desktop running, `kubectl`, `kind`, Python 3 with Pillow, ReportLab and Matplotlib, and internet access (base image, metrics-server manifest). While the script runs it prints live progress, including the autoscaler's replica count every few seconds.

4.5 Measurement method

Quantity	How it is measured
Availability	A background thread requests `/version` through the NodePort Service every 0.1 s (0.5 s during the load test); success means HTTP 200. Availability = successful / total requests.
Latency	Time per successful request; p50, p95 and maximum are reported.
Version transition	The probe records which version answered; a pod sampler records ready / not-ready / terminating pods per version every 1.5 s.
Time to scale	Seconds from `kubectl scale` until all desired pods are Ready.
CPU utilisation	`kubectl top` (millicores) and the HPA's currentMetrics (percent of the 100m request), sampled every ~9 s.

5. Source Code

All files needed to reproduce the experiments are listed with line numbers: the application and image (5.1–5.2), the cluster and manifests (5.3–5.7) and the automation script (5.8, 339 lines). Lines longer than 118 characters are wrapped and continued with "↳".

5.1 `app.py` — versioned test service

```
"""Small versioned web service used for the scaling / rolling-update experiments."""
import os
import socket
import time

from flask import Flask, jsonify, request

VERSION = os.getenv("APP_VERSION", "1.0")
BROKEN = os.getenv("APP_BROKEN", "0") == "1"          # a faulty release never becomes ready
POD = os.getenv("POD_NAME", socket.gethostname())
NODE = os.getenv("NODE_NAME", "unknown")
COLORS = {"1": "#1F5FA8", "2": "#1E8E4E", "3": "#C0392B"}
app = Flask(__name__)


@app.get("/")
def home():
    color = COLORS.get(VERSION.split(".")[0], "#444")
    return (f"<body style='font-family:sans-serif;text-align:center;margin:0;padding-
↳ top:90px;background:{color};color:white'>"
            f"<h1 style='font-size:64px;margin:0'>Version {VERSION}</h1>"
            f"<p style='font-size:22px'>Pod <b>{POD}</b></p><p style='font-size:18px'>Node {NODE}</p></body>")


@app.get("/version")
def version():
    return jsonify(version=VERSION, pod=POD, node=NODE)


@app.get("/health")                                    # liveness: process is alive
def health():
    return jsonify(status="ok")


@app.get("/ready")                                     # readiness: release is healthy
def ready():
    return (jsonify(status="not ready", version=VERSION), 503) if BROKEN else jsonify(status="ready", version=VERSION)


@app.get("/work")                                      # burns CPU for `ms` milliseconds (load generator target)
def work():
    ms = min(int(request.args.get("ms", 50)), 2000)
    end, n = time.perf_counter() + ms / 1000.0, 0
    while time.perf_counter() < end:
        n += 1
    return jsonify(version=VERSION, pod=POD, burned_ms=ms)
```

5.2 `Dockerfile` — image definition (VERSION / BROKEN build arguments)

```
FROM python:3.12-slim
ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /app
RUN pip install flask gunicorn
RUN useradd -u 10001 -m appuser
COPY app.py .
ARG VERSION=1.0
ARG BROKEN=0
ENV APP_VERSION=$VERSION APP_BROKEN=$BROKEN
USER 10001
EXPOSE 8000
CMD ["gunicorn", "-b", "0.0.0.0:8000", "-w", "2", "--access-logfile", "-", "app:app"]
```

5.3 `kind-config-task14.yaml` — Kind cluster definition

```yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    extraPortMappings:
      - containerPort: 30100   # NodePort of the web service
        hostPort: 8100         # http://localhost:8100
        protocol: TCP
  - role: worker
  - role: worker
```

5.4 `00-namespace.yaml` — Namespace

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: scale-demo
  labels:
    project: task14
```

5.5 `01-deployment.yaml` — Deployment (rolling-update strategy)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: scale-demo
  labels:
    app: web
spec:
  replicas: 3
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 60          # a stuck rollout is reported as failed after 60 s
  selector:
    matchLabels:
      app: web
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1                      # at most 1 extra pod during an update
      maxUnavailable: 0                # never drop below the desired number of ready pods
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: scale-app:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
          env:
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            - name: NODE_NAME
              valueFrom:
                fieldRef:
                  fieldPath: spec.nodeName
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 200m
              memory: 128Mi
          readinessProbe:
            httpGet:
              path: /ready
              port: 8000
            periodSeconds: 2
          livenessProbe:
            httpGet:
              path: /health
              port: 8000
            periodSeconds: 10
```

5.6 `02-service.yaml` — NodePort Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: scale-demo
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 8000
      nodePort: 30100
```

5.7 `hpa.yaml` — HorizontalPodAutoscaler

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web
  namespace: scale-demo
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 3
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50          # percent of the 100m CPU request
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
        - type: Pods
          value: 3
          periodSeconds: 15
    scaleDown:
      stabilizationWindowSeconds: 30      # shortened from the 300 s default so the demo finishes quickly
      policies:
        - type: Pods
          value: 3
          periodSeconds: 15
```

5.8 `deploy_scaling.py` — experiment automation (lines 1–62)

```
#!/usr/bin/env python3
"""Task 14 - Kubernetes scaling and rolling updates (Kind).

    python deploy_scaling.py            # run every experiment (about 15-20 minutes)
    python deploy_scaling.py --cleanup  # ... and delete the cluster afterwards

Experiments (all measured while a background probe checks availability through the Service):
  1. Manual scaling 3 -> 6 -> 2 -> 3 with timing
  2. Rolling update 1.0 -> 2.0 (maxSurge 1, maxUnavailable 0)
  3. Faulty release 3.0 (never ready): rollout stalls, old pods keep serving, rollback with 'rollout undo'
  4. Resource utilisation (metrics-server) and CPU-based autoscaling with a HorizontalPodAutoscaler under load
Every command + real output goes to evidence/run_log.json, all measurements to evidence/results.json.
"""
import argparse, json, os, shutil, statistics, subprocess, sys, threading, time, urllib.error, urllib.request
from collections import Counter
from pathlib import Path

ROOT = Path(__file__).parent
EVID = ROOT / "evidence"; EVID.mkdir(exist_ok=True)
NS, CLUSTER = "scale-demo", "task14"
BASE = "http://localhost:8100"
BASES = ["python:3.12-slim", "public.ecr.aws/docker/library/python:3.12-slim"]
MS_URL = "https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml"
LOG, RES = [], {}
BASH = "/bin/bash" if os.path.exists("/bin/bash") else None
T0 = time.time()


# ---------------------------------------------------------------------- helpers
def sh(cmd, timeout=900):
    p = subprocess.run(cmd, shell=True, capture_output=True, text=True, timeout=timeout, executable=BASH)
    return p.returncode, (p.stdout + p.stderr).strip()


def record(title, cmd, output, rc=0):
    print(f"\n=== {title} ===\n$ {cmd}\n{output}", flush=True)
    LOG.append({"title": title, "cmd": cmd, "output": output, "rc": rc})


def run(title, cmd, check=True, timeout=900):
    rc, out = sh(cmd, timeout)
    record(title, cmd, out, rc)
    if check and rc != 0:
        save(); sys.exit(f"Command failed ({rc}): {cmd}")
    return out


def run_live(title, cmd, timeout=3600, keep=14, check=True):
    print(f"\n=== {title} ===\n$ {cmd}", flush=True)
    p = subprocess.Popen(cmd, shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True,
↳ executable=BASH)
    lines, t0 = [], time.time()
    for line in p.stdout:
        print(line, end="", flush=True); lines.append(line.rstrip())
        if time.time() - t0 > timeout:
            p.kill(); break
    p.wait()
    LOG.append({"title": title, "cmd": cmd, "output": "\n".join(lines[-keep:]), "rc": p.returncode})
    if p.returncode != 0:
        if not check:
            LOG.pop(); return p.returncode
        save(); sys.exit(f"Command failed ({p.returncode}): {cmd}")
    return 0
```

5.8 `deploy_scaling.py` — experiment automation (lines 63–124)

```
63
64
65  def save():
66      (EVID / "run_log.json").write_text(json.dumps(LOG, indent=2))
67      (EVID / "results.json").write_text(json.dumps(RES, indent=2))
68      with open(EVID / "run_log.txt", "w") as f:
69          for e in LOG:
70              f.write(f"=== {e['title']} ===\n$ {e['cmd']}\n{e['output']}\n\n")
71
72
73  def jget(args):
74      rc, out = sh(f"kubectl {args} -o json")
75      try:
76          return json.loads(out)
77      except Exception:
78          return {}
79
80
81  def pods():
82      """Per-pod state of the web Deployment: version (image tag), readiness, terminating."""
83      res = []
84      for p in jget(f"get pods -n {NS} -l app=web").get("items", []):
85          st = p.get("status", {})
86          ready = any(c.get("type") == "Ready" and c.get("status") == "True" for c in st.get("conditions", []))
87          res.append({"name": p["metadata"]["name"], "version": p["spec"]["containers"][0]["image"].split(":")[-1],
88                      "ready": ready, "terminating": "deletionTimestamp" in p["metadata"], "phase": st.get("phase")})
89      return res
90
91
92  def wait_ready(n, timeout=240):
93      t0 = time.time()
94      while time.time() - t0 < timeout:
95          ps = [p for p in pods() if not p["terminating"]]
96          if len(ps) == n and all(p["ready"] for p in ps):
97              return round(time.time() - t0, 1)
98          time.sleep(1)
99      return None
100
101
102 class Probe(threading.Thread):
103     """Calls GET /version through the NodePort Service in a loop and records every result."""
104     def __init__(self, interval=0.1, path="/version"):
105         super().__init__(daemon=True); self.interval, self.path, self.stop_flag, self.rows = interval, path, False, []
106
107     def run(self):
108         while not self.stop_flag:
109             t = time.time(); status, ver = 0, None
110             try:
111                 r = urllib.request.urlopen(BASE + self.path, timeout=3)
112                 status = r.status; ver = json.loads(r.read().decode()).get("version")
113             except urllib.error.HTTPError as e:
114                 status = e.code
115             except Exception:
116                 status = 0
117             self.rows.append((t, status, ver, time.time() - t))
118             time.sleep(self.interval)
119
120     def stop(self):
121         self.stop_flag = True; self.join(timeout=10)
122
123     def summary(self):
124         n = len(self.rows); ok = sum(1 for r in self.rows if r[1] == 200)
```

5.8 `deploy_scaling.py` — experiment automation (lines 125–186)

```
125         lat = sorted(r[3] * 1000 for r in self.rows if r[1] == 200) or [0]
126         return {"requests": n, "ok": ok, "failed": n - ok, "availability_pct": round(100 * ok / n, 3) if n else 0,
127                 "versions": dict(Counter(r[2] for r in self.rows if r[1] == 200)),
128                 "latency_ms": {"p50": round(statistics.median(lat), 1), "p95": round(lat[int(0.95 * (len(lat) - 1))],
    ↳ 1), "max": round(lat[-1], 1)}}
129
130
131 def probe_text(s):
132     return (f"requests sent      : {s['requests']}\nsuccessful (HTTP 200): {s['ok']}\nfailed            :
    ↳ {s['failed']}\n"
133             f"availability       : {s['availability_pct']} %\nresponses by version: {s['versions']}\n"
134             f"latency (ms)       : p50 {s['latency_ms']['p50']}  p95 {s['latency_ms']['p95']}  max
    ↳ {s['latency_ms']['max']}")
135
136
137 class Timeline(threading.Thread):
138     """Samples the pod set every `interval` seconds."""
139     def __init__(self, interval=1.5):
140         super().__init__(daemon=True); self.interval, self.stop_flag, self.rows, self.t0 = interval, False, [],
    ↳ time.time()
141
142     def run(self):
143         while not self.stop_flag:
144             ps = pods(); row = {"t": round(time.time() - self.t0, 1)}
145             for v in sorted({p["version"] for p in ps}):
146                 row[f"v{v}_ready"] = sum(1 for p in ps if p["version"] == v and p["ready"] and not p["terminating"])
147                 row[f"v{v}_notready"] = sum(1 for p in ps if p["version"] == v and not p["ready"] and not
    ↳ p["terminating"])
148             row["terminating"] = sum(1 for p in ps if p["terminating"])
149             row["total"] = len(ps); self.rows.append(row); time.sleep(self.interval)
150
151     def stop(self):
152         self.stop_flag = True; self.join(timeout=15)
153
154
155 def load_image(tag):
156     rc, out = sh(f"kind load docker-image {tag} --name {CLUSTER}")
157     if rc != 0:     # Docker Desktop's containerd image store can break 'docker-image' loading
158         rc, out = sh(f"docker save {tag} -o /tmp/scale-app.tar && kind load image-archive /tmp/scale-app.tar --name
    ↳ {CLUSTER}")
159     return rc, out
160
161
162 def top_pods():
163     rc, out = sh(f"kubectl top pods -n {NS} --no-headers")
164     vals = []
165     for l in out.splitlines():
166         f = l.split()
167         if len(f) >= 3 and f[1].endswith("m"):
168             vals.append(int(f[1][:-1]))
169     return out, vals
170
171
172 # ---------------------------------------------------------------------- experiments
173 def exp_scaling():
174     rows = []
175     for target in (6, 2, 3):
176         cur = len([p for p in pods() if not p["terminating"]])
177         t0 = time.time(); rc, out = sh(f"kubectl scale deployment/web --replicas={target} -n {NS}")
178         secs = wait_ready(target)
179         rows.append({"from": cur, "to": target, "seconds_to_ready": secs})
180         _, lst = sh(f"kubectl get pods -n {NS} -o wide")
181         record(f"Scale {cur} to {target} replicas", f"kubectl scale deployment/web --replicas={target} -n {NS}",
182                f"{out}\nall {target} pods Ready after {secs} s\n\n{lst}")
183     RES["scaling"] = rows
184
185
186 def exp_rolling_update():
```

5.8 `deploy_scaling.py` — experiment automation (lines 187–248)

```
187      probe, tl = Probe(), Timeline(); probe.start(); tl.start(); time.sleep(3)
188      t0 = time.time()
189      sh(f"kubectl annotate deployment/web kubernetes.io/change-cause='update image to scale-app:2.0' -n {NS}
     ↳ --overwrite")
190      rc, out1 = sh(f"kubectl set image deployment/web web=scale-app:2.0 -n {NS}")
191      rc, out2 = sh(f"kubectl rollout status deployment/web -n {NS} --timeout=240s")
192      dur = round(time.time() - t0, 1); time.sleep(3); probe.stop(); tl.stop()
193      record("Rolling update to 2.0", f"kubectl set image deployment/web web=scale-app:2.0 -n {NS}",
     ↳ f"{out1}\n{out2}\n(update took {dur} s)", rc)
194      run("Rollout history", f"kubectl rollout history deployment/web -n {NS}")
195      run("ReplicaSets after update", f"kubectl get rs -n {NS}")
196      s = probe.summary(); record("Availability during rolling update", f"probe: GET {BASE}/version every 0.1 s",
     ↳ probe_text(s))
197      rows, last = [], None
198      for r in tl.rows:
199          key = tuple((k, v) for k, v in r.items() if k not in ("t",))
200          if key != last: rows.append(r); last = key
201      all_keys = sorted({k for r in rows for k in r if k != "t"})
202      txt = "t(s)    " + " ".join(f"{k:>12}" for k in all_keys) + "\n" + "\n".join(f"{r['t']:>5}  " + "
     ↳ ".join(f"{r.get(k, 0):>12}" for k in all_keys) for r in rows)
203      record("Version transition timeline", "kubectl get pods (sampled every 1.5 s)", txt)
204      RES["rolling_update"] = {"duration_s": dur, "summary": s, "timeline": tl.rows}
205
206
207  def exp_bad_release():
208      probe, tl = Probe(), Timeline(); probe.start(); tl.start(); time.sleep(3)
209      sh(f"kubectl annotate deployment/web kubernetes.io/change-cause='update image to scale-app:3.0 (faulty)' -n {NS}
     ↳ --overwrite")
210      rc0, out1 = sh(f"kubectl set image deployment/web web=scale-app:3.0 -n {NS}")
211      rc, out2 = sh(f"kubectl rollout status deployment/web -n {NS} --timeout=120s")
212      record("Faulty release 3.0 (rollout stalls)", f"kubectl set image deployment/web web=scale-app:3.0 -n {NS}",
     ↳ f"{out1}\n{out2}", rc)
213      run("Pods during failed rollout", f"kubectl get pods -n {NS} -o wide")
214      run("Deployment conditions", f"kubectl get deployment web -n {NS} -o jsonpath='{{range
     ↳ .status.conditions[*]}}{{.type}}={{.status}} ({{.reason}}){{\"\\n\"}}{{end}}'")
215      t0 = time.time()
216      rc, out3 = sh(f"kubectl rollout undo deployment/web -n {NS}")
217      rc, out4 = sh(f"kubectl rollout status deployment/web -n {NS} --timeout=180s")
218      dur = round(time.time() - t0, 1); time.sleep(3); probe.stop(); tl.stop()
219      record("Rollback with rollout undo", f"kubectl rollout undo deployment/web -n {NS}", f"{out3}\n{out4}\n(rollback
     ↳ took {dur} s)", rc)
220      run("Pods after rollback", f"kubectl get pods -n {NS} -o wide")
221      run("Rollout history after rollback", f"kubectl rollout history deployment/web -n {NS}")
222      s = probe.summary(); record("Availability during faulty release and rollback", f"probe: GET {BASE}/version every
     ↳ 0.1 s", probe_text(s))
223      RES["bad_release"] = {"rollback_s": dur, "stalled": rc0 == 0 and "successfully" not in out2, "summary": s,
     ↳ "timeline": tl.rows}
224
225
226  def exp_resources_and_hpa(load_s=150, cooldown_max=240):
227      out, vals = top_pods()
228      record("Pod resource usage (baseline)", f"kubectl top pods -n {NS}", out)
229      run("Node resource usage (baseline)", "kubectl top nodes", check=False)
230      RES["baseline_pod_cpu_m"] = vals
231      run("Apply HPA", "kubectl apply -f k8s14/optional/hpa.yaml")
232      probe = Probe(interval=0.5); probe.start()
233      stop = threading.Event()
234      def hammer():
235          while not stop.is_set():
236              try: urllib.request.urlopen(BASE + "/work?ms=100", timeout=15).read()
237              except Exception: time.sleep(0.2)
238      workers = [threading.Thread(target=hammer, daemon=True) for _ in range(14)]
239      tl, t0, load_end, captured = [], time.time(), None, False
240      for w in workers: w.start()
241      print(f"\n--- generating CPU load for {load_s} s ---", flush=True)
242      phase = "load"
243      while True:
244          t = time.time() - t0
245          if phase == "load" and t >= load_s:
246              stop.set(); phase = "cooldown"; load_end = round(t, 1)
247              print("--- load stopped, waiting for scale-down ---", flush=True)
248          h = jget(f"get hpa web -n {NS}").get("status", {})
```

5.8 `deploy_scaling.py` — experiment automation (lines 249–310)

```
        cur = h.get("currentReplicas"); des = h.get("desiredReplicas")
        cpu = None
        for m in h.get("currentMetrics", []) or []:
            if m.get("type") == "Resource": cpu = m["resource"]["current"].get("averageUtilization")
        ps = [p for p in pods() if not p["terminating"]]
        _, v = top_pods()
        tl.append({"t": round(t, 1), "phase": phase, "hpa_current": cur, "hpa_desired": des, "cpu_util_pct": cpu,
                   "pods": len(ps), "ready": sum(1 for p in ps if p["ready"]), "avg_cpu_m": round(sum(v) / len(v)) if
↳ v else None})
        print(f"t={t:5.0f}s {phase:8s} replicas={cur}->{des} cpu={cpu}% pods={len(ps)}
↳ avg_cpu={tl[-1]['avg_cpu_m']}m", flush=True)
        if phase == "load" and not captured and t >= load_s * 0.6:
            captured = True
            run("Pod resource usage (under load)", f"kubectl top pods -n {NS}", check=False)
            run("HPA status (under load)", f"kubectl get hpa web -n {NS}")
            run("Node resource usage (under load)", "kubectl top nodes", check=False)
        if phase == "cooldown" and (tl[-1]["pods"] <= 3 or t - load_end > cooldown_max):
            break
        time.sleep(6)
    probe.stop(); s = probe.summary()
    run("HPA status (after load)", f"kubectl get hpa web -n {NS}")
    run("Describe HPA", f"kubectl describe hpa web -n {NS} | tail -n 28")
    record("Availability during autoscaling under load", f"probe: GET {BASE}/version every 0.5 s", probe_text(s))
    run("Pods after autoscaling", f"kubectl get pods -n {NS}")
    RES["hpa"] = {"timeline": tl, "load_seconds": load_end, "peak_pods": max(r["pods"] for r in tl), "summary": s,
                  "min_replicas": 3, "max_replicas": 6, "target_cpu_pct": 50}


def main():
    ap = argparse.ArgumentParser(); ap.add_argument("--cleanup", action="store_true"); a = ap.parse_args()
    for t in ("docker", "kubectl", "kind"):
        if not shutil.which(t):
            sys.exit(f"'{t}' not found in PATH - install it first (see README.md)")
    run("Tool versions", "docker --version && kubectl version --client && kind version")

    # 1. images: 1.0 (initial), 2.0 (update) and 3.0 (faulty release)
    for ver, bad, label in (("1.0", 0, "Build image 1.0"), ("2.0", 0, "Build image 2.0"), ("3.0", 1, "Build image 3.0
↳ (faulty)")):
        for n, base in enumerate(BASES):               # Docker Hub sometimes fails (auth 500): fall back to an AWS
↳ mirror of the same image
            rc = run_live(label, f"docker build --progress=plain --provenance=false --sbom=false --build-arg
↳ BASE={base} "
                                 f"--build-arg VERSION={ver} --build-arg BROKEN={bad} -t scale-app:{ver} app14 2>&1",
↳ check=(n == len(BASES) - 1))
            if rc == 0:
                break
            print(f"\n!! build with base image {base} failed - trying the next mirror ...", flush=True)
    run("Docker images", "docker image ls scale-app")

    # 2. cluster
    existing = run("List kind clusters", "kind get clusters", check=False)
    if CLUSTER not in existing.split():
        run("Create Kind cluster (1 control-plane + 2 workers)", f"kind create cluster --name {CLUSTER} --config kind-
↳ config-task14.yaml --wait 120s")
    run("Set kubectl context", f"kubectl config use-context kind-{CLUSTER}")
    run("Wait for all nodes Ready", "kubectl wait --for=condition=Ready nodes --all --timeout=180s")
    run("Cluster nodes", "kubectl get nodes -o wide")
    for ver in ("1.0", "2.0", "3.0"):
        rc, out = load_image(f"scale-app:{ver}")
        record(f"Load image {ver} into Kind", f"kind load docker-image scale-app:{ver} --name {CLUSTER}", out[-300:],
↳ rc)
        if rc != 0: save(); sys.exit("Could not load image into Kind")

    # 3. metrics-server (needed for kubectl top and the HPA)
    run("Install metrics-server", f"kubectl apply -f {MS_URL}", check=False)
    run("Patch metrics-server for Kind", "kubectl patch deployment metrics-server -n kube-system --type=json "
        "-p='[{\"op\":\"add\",\"path\":\"/spec/template/spec/containers/0/args/-\",\"value\":\"--kubelet-insecure-
↳ tls\"}]'", check=False)
    run("Wait for metrics-server", "kubectl rollout status deployment/metrics-server -n kube-system --timeout=180s",
↳ check=False)

    # 4. application
```

5.8 `deploy_scaling.py` — experiment automation (lines 311–339)

```
311      run("Apply manifests", "kubectl apply -f k8s14/")
312      run("Wait for initial rollout", f"kubectl rollout status deployment/web -n {NS} --timeout=240s")
313      run("Get pods (initial)", f"kubectl get pods -n {NS} -o wide")
314      run("Get deployment and service", f"kubectl get deploy,svc,endpointslices -n {NS}")
315      ok = None
316      for _ in range(20):
317          try: ok = urllib.request.urlopen(BASE + "/version", timeout=5).read().decode(); break
318          except Exception: time.sleep(2)
319      record("Application reachable (version 1.0)", f"curl -s {BASE}/version", ok or "FAILED to reach the application",
     ↳ 0 if ok else 1)
320
321      # 5. experiments
322      exp_scaling()
323      exp_rolling_update()
324      exp_bad_release()
325      for _ in range(12):                                  # metrics-server needs a moment before 'kubectl top' works
326          if sh(f"kubectl top pods -n {NS}")[0] == 0: break
327          time.sleep(10)
328      exp_resources_and_hpa()
329
330      run("Deployment details", f"kubectl describe deployment web -n {NS}")
331      run("Recent events", f"kubectl get events -n {NS} --sort-by=.lastTimestamp | tail -n 40")
332      save()
333      print(f"\nDone. Logs + measurements saved in {EVID}  (total {round((time.time()-T0)/60,1)} min)")
334      if a.cleanup:
335          subprocess.run(f"kind delete cluster --name {CLUSTER}", shell=True)
336
337
338  if __name__ == "__main__":
339      main()
```

6. Screenshots and Command Output

Terminal figures show the exact output captured by `deploy_scaling.py` during the run (`evidence/run_log.json`), rendered in a terminal style rather than retyped. Charts in Section 7 are drawn from the measurements in `evidence/results.json`. Pod names, IPs and timings are those of this particular run.

6.1 Images and cluster

Tool versions

```
$ docker --version && kubectl version --client && kind version
Docker version 29.8.2, build 7fc2dff
Client Version: v1.37.1
Kustomize Version: v5.8.1
kind v0.33.0 go1.27.0 darwin/arm64
```

Build image 1.0

```
$ docker build --progress=plain --provenance=false --sbom=false --build-arg BASE=python:3.12-slim --build-arg VERSION=1.0 --build-arg BROKEN=0 -t scale-app:1.0 app14 2>&1
#10 [5/5] COPY app.py .
#10 DONE 0.0s

#11 exporting to image
#11 exporting layers
#11 exporting layers 0.7s done
#11 exporting manifest sha256:a4c99e89918dacfbbab5410128d059e9c7d4900781b05009139c826cb10ed9ef done
#11 exporting config sha256:39b76e757e21ba319a9d83616c5b8c9732d3e9d1cbaa85eb16e414a657c20d54 done
#11 naming to docker.io/library/scale-app:1.0 done
#11 unpacking to docker.io/library/scale-app:1.0
#11 unpacking to docker.io/library/scale-app:1.0 0.2s done
#11 DONE 0.9s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/2xspyq5ge1b9029kzk2bbn2tl
```

Docker images

```
$ docker image ls scale-app
IMAGE           ID             DISK USAGE   CONTENT SIZE   EXTRA
scale-app:1.0   a4c99e89918d        226MB         49.1MB
scale-app:2.0   035ee86bf47f        226MB         49.1MB
scale-app:3.0   09fb620cbe44        226MB         49.1MB
```

Figure 1: Tool versions, build of image scale-app:1.0 (last lines) and the images 1.0, 2.0 and the faulty 3.0 that were built from the same Dockerfile.

List kind clusters

```
$ kind get clusters
No kind clusters found.
```

Create Kind cluster (1 control-plane + 2 workers)

```
$ kind create cluster --name task14 --config kind-config-task14.yaml --wait 120s
Creating cluster "task14" ...
 • Ensuring node image (kindest/node:v1.37.0) □□  ...
 ✓ Ensuring node image (kindest/node:v1.37.0) □□
 • Preparing nodes □ □ □   ...
 ✓ Preparing nodes □ □ □
 • Writing configuration □  ...
 ✓ Writing configuration □
 • Starting control-plane □□  ...
 ✓ Starting control-plane □□
 • Installing CNI □  ...
 ✓ Installing CNI □
 • Installing StorageClass □  ...
 ✓ Installing StorageClass □
 • Joining worker nodes □  ...
 ✓ Joining worker nodes □
 • Waiting ≤ 2m0s for control-plane = Ready □  ...
 ✓ Waiting ≤ 2m0s for control-plane = Ready □
 • Ready after 10s □
Set kubectl context to "kind-task14"
You can now use your cluster with:

kubectl cluster-info --context kind-task14

Not sure what to do next? □  Check out https://kind.sigs.k8s.io/docs/user/quick-start/
```

Set kubectl context

```
$ kubectl config use-context kind-task14
Switched to context "kind-task14".
```

Wait for all nodes Ready

```
$ kubectl wait --for=condition=Ready nodes --all --timeout=180s
node/task14-control-plane condition met
node/task14-worker condition met
node/task14-worker2 condition met
```

Cluster nodes

```
$ kubectl get nodes -o wide
NAME                 STATUS   ROLES           AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                       KERNEL-VERSION
    CONTAINER-RUNTIME
task14-control-plane   Ready    control-plane   23s   v1.37.0   172.18.0.2    <none>        Debian GNU/Linux 13 (trixie)   7.0.14-linuxkit (arm64)
    containerd://2.3.4
task14-worker          Ready    <none>          13s   v1.37.0   172.18.0.4    <none>        Debian GNU/Linux 13 (trixie)   7.0.14-linuxkit (arm64)
    containerd://2.3.4
task14-worker2         Ready    <none>          13s   v1.37.0   172.18.0.3    <none>        Debian GNU/Linux 13 (trixie)   7.0.14-linuxkit (arm64)
    containerd://2.3.4
```

Figure 2: Creation of the three-node Kind cluster task14 (host port 8100 mapped to NodePort 30100), context, node readiness and node list.

```
Load image 1.0 into Kind
$ kind load docker-image scale-app:1.0 --name task14
app:1.0" with ID "sha256:a4c99e89918dacfbbab5410128d059e9c7d4900781b05009139c826cb10ed9ef" not yet present on node "task14-worker2", loading...
Image: "scale-app:1.0" with ID "sha256:a4c99e89918dacfbbab5410128d059e9c7d4900781b05009139c826cb10ed9ef" not yet present on node "task14-worker",
    loading...
Load image 2.0 into Kind
$ kind load docker-image scale-app:2.0 --name task14
app:2.0" with ID "sha256:035ee86bf47fbf4a3e2a2d87f2e4864e1e425e97f28b062ba0d5c919d61f7a9c" not yet present on node "task14-worker2", loading...
Image: "scale-app:2.0" with ID "sha256:035ee86bf47fbf4a3e2a2d87f2e4864e1e425e97f28b062ba0d5c919d61f7a9c" not yet present on node "task14-worker",
    loading...
Load image 3.0 into Kind
$ kind load docker-image scale-app:3.0 --name task14
app:3.0" with ID "sha256:09fb620cbe446acf6ae989d20c6136822cd4903cfb8350d8491b69c6f3c0d72d" not yet present on node "task14-worker2", loading...
Image: "scale-app:3.0" with ID "sha256:09fb620cbe446acf6ae989d20c6136822cd4903cfb8350d8491b69c6f3c0d72d" not yet present on node "task14-worker",
    loading...
Install metrics-server
$ kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
serviceaccount/metrics-server created
clusterrole.rbac.authorization.k8s.io/system:aggregated-metrics-reader created
clusterrole.rbac.authorization.k8s.io/system:metrics-server created
rolebinding.rbac.authorization.k8s.io/metrics-server-auth-reader created
clusterrolebinding.rbac.authorization.k8s.io/metrics-server:system:auth-delegator created
clusterrolebinding.rbac.authorization.k8s.io/system:metrics-server created
service/metrics-server created
deployment.apps/metrics-server created
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created
Patch metrics-server for Kind
$ kubectl patch deployment metrics-server -n kube-system --type=json -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'
deployment.apps/metrics-server patched
Wait for metrics-server
$ kubectl rollout status deployment/metrics-server -n kube-system --timeout=180s
Waiting for deployment "metrics-server" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "metrics-server" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "metrics-server" rollout to finish: 1 old replicas are pending termination...
deployment "metrics-server" successfully rolled out
```

Figure 3: The three images loaded into the Kind nodes and metrics-server installed and patched (needed for kubectl top and the autoscaler).

6.2 Initial deployment

```
Apply manifests
$ kubectl apply -f k8s14/
namespace/scale-demo created
deployment.apps/web created
service/web created
Wait for initial rollout
$ kubectl rollout status deployment/web -n scale-demo --timeout=240s
Waiting for deployment "web" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "web" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "web" rollout to finish: 2 of 3 updated replicas are available...
deployment "web" successfully rolled out
Get pods (initial)
$ kubectl get pods -n scale-demo -o wide
NAME                   READY   STATUS    RESTARTS   AGE   IP           NODE            NOMINATED NODE   READINESS GATES
web-76cf8458c9-6tx64   1/1     Running   0          3s    10.244.1.3   task14-worker2   <none>           <none>
web-76cf8458c9-8dcmq   1/1     Running   0          3s    10.244.2.4   task14-worker    <none>           <none>
web-76cf8458c9-bttdp   1/1     Running   0          3s    10.244.2.3   task14-worker    <none>           <none>
Get deployment and service
$ kubectl get deploy,svc,endpointslices -n scale-demo
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/web   3/3     3            3           3s

NAME          TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
service/web   NodePort   10.96.16.135   <none>        80:30100/TCP   3s

NAME                                    ADDRESSTYPE   PORTS   ENDPOINTS                         AGE
endpointslice.discovery.k8s.io/web-4tw42   IPv4          8000    10.244.2.4,10.244.2.3,10.244.1.3   3s
Application reachable (version 1.0)
$ curl -s http://localhost:8100/version
{"node":"task14-worker","pod":"web-76cf8458c9-8dcmq","version":"1.0"}
```

Figure 4: Manifests applied, the initial 3-replica rollout, pods, Deployment, Service and EndpointSlice, and a first request to the application.

6.3 Scaling replicas

Scale 3 to 6 replicas

```
$ kubectl scale deployment/web --replicas=6 -n scale-demo
deployment.apps/web scaled
all 6 pods Ready after 5.6 s

NAME                   READY   STATUS    RESTARTS   AGE   IP           NODE             NOMINATED NODE   READINESS GATES
web-76cf8458c9-4n6z4   1/1     Running   0          5s    10.244.2.5   task14-worker    <none>           <none>
web-76cf8458c9-6tx64   1/1     Running   0          9s    10.244.1.3   task14-worker2   <none>           <none>
web-76cf8458c9-8dcmq   1/1     Running   0          9s    10.244.2.4   task14-worker    <none>           <none>
web-76cf8458c9-bttdp   1/1     Running   0          9s    10.244.2.3   task14-worker    <none>           <none>
web-76cf8458c9-mkdv5   1/1     Running   0          5s    10.244.1.4   task14-worker2   <none>           <none>
web-76cf8458c9-mrj6x   1/1     Running   0          5s    10.244.1.5   task14-worker2   <none>           <none>
```

Scale 6 to 2 replicas

```
$ kubectl scale deployment/web --replicas=2 -n scale-demo
deployment.apps/web scaled
all 2 pods Ready after 0.1 s

NAME                   READY   STATUS        RESTARTS   AGE   IP           NODE             NOMINATED NODE   READINESS GATES
web-76cf8458c9-4n6z4   1/1     Terminating   0          6s    10.244.2.5   task14-worker    <none>           <none>
web-76cf8458c9-6tx64   1/1     Running       0          10s   10.244.1.3   task14-worker2   <none>           <none>
web-76cf8458c9-8dcmq   1/1     Running       0          10s   10.244.2.4   task14-worker    <none>           <none>
web-76cf8458c9-bttdp   1/1     Terminating   0          10s   10.244.2.3   task14-worker    <none>           <none>
web-76cf8458c9-mkdv5   1/1     Terminating   0          6s    10.244.1.4   task14-worker2   <none>           <none>
web-76cf8458c9-mrj6x   1/1     Terminating   0          6s    10.244.1.5   task14-worker2   <none>           <none>
```

Scale 2 to 3 replicas

```
$ kubectl scale deployment/web --replicas=3 -n scale-demo
deployment.apps/web scaled
all 3 pods Ready after 4.5 s

NAME                   READY   STATUS    RESTARTS   AGE   IP           NODE             NOMINATED NODE   READINESS GATES
web-76cf8458c9-4tx79   1/1     Running   0          4s    10.244.2.6   task14-worker    <none>           <none>
web-76cf8458c9-6tx64   1/1     Running   0          14s   10.244.1.3   task14-worker2   <none>           <none>
web-76cf8458c9-8dcmq   1/1     Running   0          14s   10.244.2.4   task14-worker    <none>           <none>
```

Figure 5: Manual scaling 3 → 6 → 2 → 3 with kubectl scale; each step shows the time until every pod is Ready and the resulting pod list.

6.4 Rolling update

Rolling update to 2.0

```
$ kubectl set image deployment/web web=scale-app:2.0 -n scale-demo
deployment.apps/web image updated
Waiting for deployment "web" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "web" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "web" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "web" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "web" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "web" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "web" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "web" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "web" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "web" rollout to finish: 1 old replicas are pending termination...
deployment "web" successfully rolled out
(update took 9.9 s)
```

Rollout history

```
$ kubectl rollout history deployment/web -n scale-demo
deployment.apps/web
REVISION  CHANGE-CAUSE
1         update image to scale-app:2.0
2         update image to scale-app:2.0
```

ReplicaSets after update

```
$ kubectl get rs -n scale-demo
NAME             DESIRED   CURRENT   READY   AGE
web-6ddd4dc98b   3         3         3       14s
web-76cf8458c9   0         0         0       32s
```

Figure 6: Rolling update from version 1.0 to 2.0: command and rollout progress, revision history (with change-cause) and the two ReplicaSets.

Availability during rolling update

```
$ probe: GET http://localhost:8100/version every 0.1 s
requests sent      : 145
successful (HTTP 200): 141
failed             : 4
availability       : 97.241 %
responses by version: {'1.0': 95, '2.0': 46}
latency (ms)       : p50 5.8  p95 9.2  max 29.1
```

Version transition timeline

```
$ kubectl get pods (sampled every 1.5 s)
t(s)    terminating        total v1.0_notready   v1.0_ready v2.0_notready   v2.0_ready
  0.1             0            3             0            3             0            0
  4.8             0            4             0            3             1            0
  8.0             1            5             0            2             1            1
  9.6             0            4             0            2             1            1
 11.2             0            4             0            1             1            2
 14.4             0            3             0            0             0            3
```

Figure 7: Availability measured by a probe sending 10 requests per second through the Service during the rolling update, and the pod-state timeline sampled every 1.5 s.

6.5 Faulty release and rollback

Faulty release 3.0 (rollout stalls)

```
$ kubectl set image deployment/web web=scale-app:3.0 -n scale-demo
deployment.apps/web image updated
Waiting for deployment "web" rollout to finish: 1 out of 3 new replicas have been updated...
error: deployment "web" exceeded its progress deadline
```

Pods during failed rollout

```
$ kubectl get pods -n scale-demo -o wide
NAME                   READY   STATUS    RESTARTS   AGE   IP           NODE             NOMINATED NODE   READINESS GATES
web-6ddd4dc98b-htr9c   1/1     Running   0          75s   10.244.1.6   task14-worker2   <none>           <none>
web-6ddd4dc98b-n2d55   1/1     Running   0          72s   10.244.2.8   task14-worker    <none>           <none>
web-6ddd4dc98b-rbmvr   1/1     Running   0          78s   10.244.2.7   task14-worker    <none>           <none>
web-bb78f4c6f-lbm6w    0/1     Running   0          61s   10.244.2.9   task14-worker    <none>           <none>
```

Deployment conditions

```
$ kubectl get deployment web -n scale-demo -o jsonpath='{range .status.conditions[*]}{.type}={.status} ({.reason}){"\n"}{end}'
Available=True (MinimumReplicasAvailable)
Progressing=False (ProgressDeadlineExceeded)
```

Figure 8: Faulty release 3.0 (never ready): the rollout stalls, new pods stay 0/1 Ready, the Deployment reports the stalled progress.

Rollback with rollout undo

```
$ kubectl rollout undo deployment/web -n scale-demo
deployment.apps/web rolled back
Warning: resource deployments/web was previously managed with 'kubectl apply'. Rolling back will not update the kubectl.kubernetes.io/last-applied-
    configuration annotation, which may cause unexpected behavior on future 'kubectl apply' operations. Consider using 'kubectl apply' with your previous
    configuration file instead.
deployment "web" successfully rolled out
(rollback took 0.2 s)
```

Pods after rollback

```
$ kubectl get pods -n scale-demo -o wide
NAME                   READY   STATUS    RESTARTS   AGE   IP           NODE             NOMINATED NODE   READINESS GATES
web-6ddd4dc98b-htr9c   1/1     Running   0          79s   10.244.1.6   task14-worker2   <none>           <none>
web-6ddd4dc98b-n2d55   1/1     Running   0          76s   10.244.2.8   task14-worker    <none>           <none>
web-6ddd4dc98b-rbmvr   1/1     Running   0          82s   10.244.2.7   task14-worker    <none>           <none>
```

Rollout history after rollback

```
$ kubectl rollout history deployment/web -n scale-demo
deployment.apps/web
REVISION  CHANGE-CAUSE
1         update image to scale-app:2.0
3         update image to scale-app:3.0 (faulty)
4         update image to scale-app:3.0 (faulty)
```

Availability during faulty release and rollback

```
$ probe: GET http://localhost:8100/version every 0.1 s
requests sent      : 607
successful (HTTP 200): 607
failed             : 0
availability       : 100.0 %
responses by version: {'2.0': 607}
latency (ms)       : p50 6.8  p95 8.8  max 18.5
```

Figure 9: Rollback with kubectl rollout undo, pods and history afterwards, and the availability measured throughout.

6.6 Resource utilisation and autoscaling

```
Pod resource usage (baseline)
$ kubectl top pods -n scale-demo
web-6ddd4dc98b-htr9c   10m   53Mi
web-6ddd4dc98b-n2d55   9m    53Mi
web-6ddd4dc98b-rbmvr   11m   53Mi
Node resource usage (baseline)
$ kubectl top nodes
NAME                   CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
task14-control-plane   175m         1%       879Mi           11%
task14-worker          67m          0%       415Mi           5%
task14-worker2         49m          0%       331Mi           4%
Apply HPA
$ kubectl apply -f k8s14/optional/hpa.yaml
horizontalpodautoscaler.autoscaling/web created
```

Figure 10: Baseline CPU/memory of pods and nodes (metrics-server) and creation of the HorizontalPodAutoscaler (3–6 pods, 50 % CPU target).

```
Pod resource usage (under load)
$ kubectl top pods -n scale-demo
NAME                   CPU(cores)   MEMORY(bytes)
web-6ddd4dc98b-4rvbd   187m         53Mi
web-6ddd4dc98b-97bk7   189m         53Mi
web-6ddd4dc98b-htr9c   185m         53Mi
web-6ddd4dc98b-n2d55   175m         54Mi
web-6ddd4dc98b-p82qz   192m         53Mi
web-6ddd4dc98b-rbmvr   187m         53Mi
HPA status (under load)
$ kubectl get hpa web -n scale-demo
NAME   REFERENCE        TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
web    Deployment/web   cpu: 185%/50%   3         6         6          94s
Node resource usage (under load)
$ kubectl top nodes
NAME                   CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
task14-control-plane   105m         1%       890Mi           11%
task14-worker          593m         5%       421Mi           5%
task14-worker2         608m         6%       452Mi           5%
```

Figure 11: Resource usage and autoscaler status while CPU load is applied.

HPA status (after load)

```
$ kubectl get hpa web -n scale-demo
NAME   REFERENCE        TARGETS        MINPODS   MAXPODS   REPLICAS   AGE
web    Deployment/web   cpu: 14%/50%   3         6         6          3m21s
```

Pods after autoscaling

```
$ kubectl get pods -n scale-demo
NAME                   READY   STATUS    RESTARTS   AGE
web-6ddd4dc98b-97bk7   1/1     Running   0          2m51s
web-6ddd4dc98b-htr9c   1/1     Running   0          4m40s
web-6ddd4dc98b-p82qz   1/1     Running   0          2m51s
```

Availability during autoscaling under load

```
$ probe: GET http://localhost:8100/version every 0.5 s
requests sent      : 349
successful (HTTP 200): 349
failed             : 0
availability       : 100.0 %
responses by version: {'2.0': 349}
latency (ms)       : p50 9.3  p95 295.3  max 498.1
```

Figure 12: Autoscaler status after the load stopped, pods after scale-down and availability during the whole autoscaling experiment.

Describe HPA

```
$ kubectl describe hpa web -n scale-demo | tail -n 28
Metrics:                                               ( current / target )
  resource cpu on pods  (as a percentage of request):  14% (14m) / 50%
Min replicas:                                          3
Max replicas:                                          6
Behavior:
  Scale Up:
    Stabilization Window: 0 seconds
    Select Policy: Max
    Policies:
      - Type: Pods  Value: 3  Period: 15 seconds
  Scale Down:
    Stabilization Window: 30 seconds
    Select Policy: Max
    Policies:
      - Type: Pods  Value: 3  Period: 15 seconds
Deployment pods:    6 current / 3 desired
Conditions:
  Type            Status  Reason              Message
  ----            ------  ------              -------
  AbleToScale     True    SucceededRescale    the HPA controller was able to update the target scale to 3
  ScalingActive   True    ValidMetricFound    the HPA was able to successfully calculate a replica count from cpu resource utilization (percentage of
    request)
  ScalingLimited  False   DesiredWithinRange  the desired count is within the acceptable range
  ScaledToZero    False   NotScaledToZero     the HPA controller did not scale the workload to zero
Events:
  Type    Reason             Age    From                       Message
  ----    ------             ----   ----                       -------
  Normal  SuccessfulRescale  2m51s  horizontal-pod-autoscaler  New size: 6; reason: cpu resource utilization (percentage of request) above target
  Normal  SuccessfulRescale  6s     horizontal-pod-autoscaler  New size: 3; reason: All metrics below target
```

Figure 13: kubectl describe hpa: conditions and the scaling events recorded by the autoscaler.

6.7 Monitoring deployment status

Deployment details

```
$ kubectl describe deployment web -n scale-demo
Name:                   web
Namespace:              scale-demo
CreationTimestamp:      Sat, 10 Oct 2026 02:48:49 +0530
Labels:                 app=web
Annotations:            deployment.kubernetes.io/revision: 4
                        kubernetes.io/change-cause: update image to scale-app:3.0 (faulty)
Selector:               app=web
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  0 max unavailable, 1 max surge
Pod Template:
  Labels:  app=web
  Containers:
   web:
    Image:      scale-app:2.0
    Port:       8000/TCP
    Host Port:  0/TCP
    Limits:
      cpu:     200m
      memory:  128Mi
    Requests:
      cpu:      100m
      memory:   64Mi
    Liveness:   http-get http://:8000/health delay=0s timeout=1s period=10s successThreshold=1 failureThreshold=3
    Readiness:  http-get http://:8000/ready delay=0s timeout=1s period=2s successThreshold=1 failureThreshold=3
    Environment:
```

Figure 14 (part 1 of 2): kubectl describe deployment web at the end of the run: strategy, probes, resources, conditions and ReplicaSets.

Deployment details

```
$ kubectl describe deployment web -n scale-demo
      POD_NAME:     (v1:metadata.name)
      NODE_NAME:    (v1:spec.nodeName)
    Mounts:         <none>
  Volumes:          <none>
  Node-Selectors:   <none>
  Tolerations:      <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Progressing    True    NewReplicaSetAvailable
  Available      True    MinimumReplicasAvailable
OldReplicaSets:  web-76cf8458c9 (0/0 replicas created), web-bb78f4c6f (0/0 replicas created)
NewReplicaSet:   web-6ddd4dc98b (3/3 replicas created)
Events:
  Type    Reason             Age                From                   Message
  ----    ------             ----               ----                   -------
  Normal  ScalingReplicaSet  5m1s               deployment-controller  Scaled up replica set web-76cf8458c9 from 0 to 3
  Normal  ScalingReplicaSet  4m57s              deployment-controller  Scaled up replica set web-76cf8458c9 from 3 to 6
  Normal  ScalingReplicaSet  4m51s              deployment-controller  Scaled down replica set web-76cf8458c9 from 6 to 2
  Normal  ScalingReplicaSet  4m51s              deployment-controller  Scaled up replica set web-76cf8458c9 from 2 to 3
  Normal  ScalingReplicaSet  4m43s              deployment-controller  Scaled up replica set web-6ddd4dc98b from 0 to 1
  Normal  ScalingReplicaSet  4m40s              deployment-controller  Scaled down replica set web-76cf8458c9 from 3 to 2
  Normal  ScalingReplicaSet  4m40s              deployment-controller  Scaled up replica set web-6ddd4dc98b from 1 to 2
  Normal  ScalingReplicaSet  4m37s              deployment-controller  Scaled down replica set web-76cf8458c9 from 2 to 1
  Normal  ScalingReplicaSet  4m37s              deployment-controller  Scaled up replica set web-6ddd4dc98b from 2 to 3
  Normal  ScalingReplicaSet  6s (x5 over 4m34s)  deployment-controller  (combined from similar events): Scaled down replica set web-6ddd4dc98b from 6
    to 3
```

Figure 14 (part 2 of 2): continued.

Recent events

```
$ kubectl get events -n scale-demo --sort-by=.lastTimestamp | tail -n 40
4m26s       Normal    SuccessfulCreate    replicaset/web-bb78f4c6f      Created pod: web-bb78f4c6f-lbm6w
4m26s       Normal    Scheduled           pod/web-bb78f4c6f-lbm6w       Successfully assigned scale-demo/web-bb78f4c6f-lbm6w to task14-worker
4m25s       Normal    Pulled              pod/web-bb78f4c6f-lbm6w       Container image "scale-app:3.0" already present on machine and can be accessed
     by the pod
4m25s       Normal    Created             pod/web-bb78f4c6f-lbm6w       Container created
4m25s       Normal    Started             pod/web-bb78f4c6f-lbm6w       Container started
4m25s       Warning   Unhealthy           pod/web-bb78f4c6f-lbm6w       Readiness probe failed: Get "http://10.244.2.9:8000/ready": dial tcp
    10.244.2.9:8000: connect: connection refused
3m39s       Warning   Unhealthy           pod/web-bb78f4c6f-lbm6w       Readiness probe failed: HTTP probe failed with statuscode: 503
3m25s       Normal    Killing             pod/web-bb78f4c6f-lbm6w       Stopping container web
3m25s       Normal    SuccessfulDelete    replicaset/web-bb78f4c6f      Deleted pod: web-bb78f4c6f-lbm6w
2m51s       Normal    Scheduled           pod/web-6ddd4dc98b-p82qz      Successfully assigned scale-demo/web-6ddd4dc98b-p82qz to task14-worker2
2m51s       Normal    SuccessfulCreate    replicaset/web-6ddd4dc98b     Created pod: web-6ddd4dc98b-p82qz
2m51s       Normal    SuccessfulCreate    replicaset/web-6ddd4dc98b     Created pod: web-6ddd4dc98b-97bk7
2m51s       Normal    Created             pod/web-6ddd4dc98b-p82qz      Container created
2m51s       Normal    Scheduled           pod/web-6ddd4dc98b-97bk7      Successfully assigned scale-demo/web-6ddd4dc98b-97bk7 to task14-worker
2m51s       Normal    SuccessfulRescale   horizontalpodautoscaler/web   New size: 6; reason: cpu resource utilization (percentage of request) above
    target
2m51s       Normal    Scheduled           pod/web-6ddd4dc98b-4rvbd      Successfully assigned scale-demo/web-6ddd4dc98b-4rvbd to task14-worker2
2m51s       Normal    Pulled              pod/web-6ddd4dc98b-p82qz      Container image "scale-app:2.0" already present on machine and can be accessed
     by the pod
2m51s       Normal    SuccessfulCreate    replicaset/web-6ddd4dc98b     Created pod: web-6ddd4dc98b-4rvbd
2m50s       Normal    Created             pod/web-6ddd4dc98b-4rvbd      Container created
2m50s       Warning   Unhealthy           pod/web-6ddd4dc98b-4rvbd      Readiness probe failed: Get "http://10.244.1.7:8000/ready": dial tcp
    10.244.1.7:8000: connect: connection refused
2m50s       Normal    Pulled              pod/web-6ddd4dc98b-4rvbd      Container image "scale-app:2.0" already present on machine and can be accessed
     by the pod
2m50s       Normal    Pulled              pod/web-6ddd4dc98b-97bk7      Container image "scale-app:2.0" already present on machine and can be accessed
     by the pod
2m50s       Normal    Started             pod/web-6ddd4dc98b-p82qz      Container started
2m50s       Warning   Unhealthy           pod/web-6ddd4dc98b-p82qz      Readiness probe failed: Get "http://10.244.1.8:8000/ready": dial tcp
    10.244.1.8:8000: connect: connection refused
2m50s       Normal    Started             pod/web-6ddd4dc98b-4rvbd      Container started
2m50s       Normal    Created             pod/web-6ddd4dc98b-97bk7      Container created
2m50s       Normal    Started             pod/web-6ddd4dc98b-97bk7      Container started
2m50s       Warning   Unhealthy           pod/web-6ddd4dc98b-97bk7      Readiness probe failed: Get "http://10.244.2.10:8000/ready": dial tcp
    10.244.2.10:8000: connect: connection refused
2m48s       Warning   Unhealthy           pod/web-6ddd4dc98b-p82qz      Readiness probe failed: Get "http://10.244.1.8:8000/ready": context deadline
    exceeded (Client.Timeout exceeded while awaiting headers)
2m48s       Warning   Unhealthy           pod/web-6ddd4dc98b-97bk7      Readiness probe failed: Get "http://10.244.2.10:8000/ready": context deadline
    exceeded (Client.Timeout exceeded while awaiting headers)
6s          Normal    Killing             pod/web-6ddd4dc98b-n2d55      Stopping container web
6s          Normal    SuccessfulDelete    replicaset/web-6ddd4dc98b     Deleted pod: web-6ddd4dc98b-4rvbd
6s          Normal    Killing             pod/web-6ddd4dc98b-4rvbd      Stopping container web
6s          Normal    SuccessfulDelete    replicaset/web-6ddd4dc98b     Deleted pod: web-6ddd4dc98b-n2d55
6s          Normal    SuccessfulDelete    replicaset/web-6ddd4dc98b     Deleted pod: web-6ddd4dc98b-rbmvr
6s          Normal    Killing             pod/web-6ddd4dc98b-rbmvr      Stopping container web
6s          Normal    ScalingReplicaSet   deployment/web                (combined from similar events): Scaled down replica set web-6ddd4dc98b from 6
    to 3
6s          Normal    SuccessfulRescale   horizontalpodautoscaler/web   New size: 3; reason: All metrics below target
5s          Warning   Unhealthy           pod/web-6ddd4dc98b-rbmvr      Readiness probe failed: Get "http://10.244.2.7:8000/ready": dial tcp
    10.244.2.7:8000: connect: connection refused
5s          Warning   Unhealthy           pod/web-6ddd4dc98b-4rvbd      Readiness probe failed: Get "http://10.244.1.7:8000/ready": read tcp
    10.244.1.1:57024->10.244.1.7:8000: read: connection reset by peer
```

Figure 15: Namespace events of the whole run (scheduling, scaling, probe failures of the faulty release, autoscaler decisions).

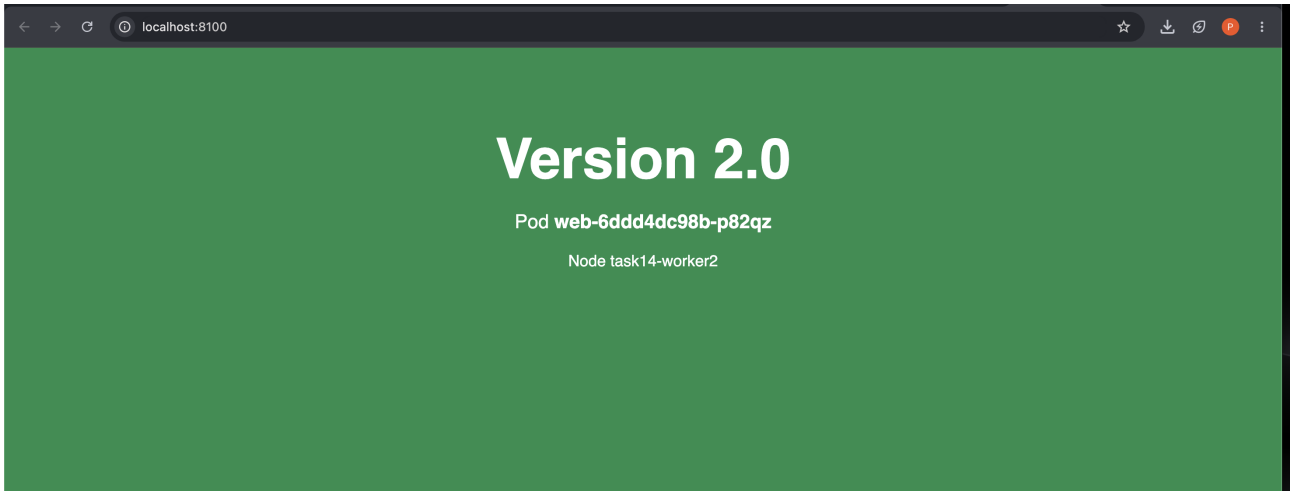


Figure 16: The application opened in a browser at http://localhost:8100 (the colour and the version number identify the release).

7. Results

The tables and charts below are computed from the measurements recorded during the run. Mapping to the deliverables: **scaling documentation** — Sections 4 and 7.1; **monitoring screenshots** — Section 6 and the charts; **performance observations** — Sections 7 and 8.

7.1 Scaling replicas

Change	Time until all pods Ready	Observation
3 → 6 replicas	5.6 s	Scale-up has to start containers and pass the readiness probe
6 → 2 replicas	0.1 s	Scale-down removes surplus pods; no new containers are started
2 → 3 replicas	4.5 s	Scale-up has to start containers and pass the readiness probe

Scaling up is limited by container start-up and the readiness probe (period 2 s); scaling down only terminates pods and the remaining pods stay Ready, so it completes as soon as the extra pods are gone.

7.2 Rolling update 1.0 → 2.0

Metric	Value
Update duration (set image → rollout complete)	9.9 s
Probe requests sent	145
Failed requests	4
Availability	97.241 %
Responses by version	1.0: 95, 2.0: 46
Latency p50 / p95 / max	5.8 / 9.2 / 29.1 ms

t (s)	terminating	v1.0 notready	v1.0 ready	v2.0 notready	v2.0 ready
0.1	0	0	3	0	0
4.8	0	0	3	1	0
8.0	1	0	2	1	1
9.6	0	0	2	1	1
11.2	0	0	1	1	2
14.4	0	0	0	0	3

Pod states during the update (a row is shown whenever the state changed). With maxSurge 1 and maxUnavailable 0 the new pod must become ready before an old pod is terminated.

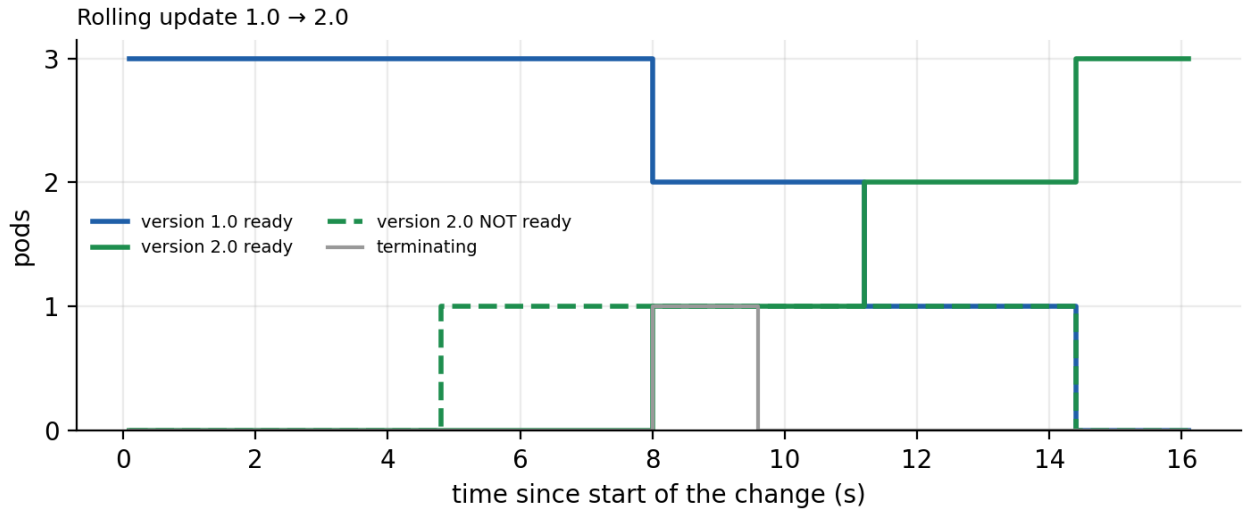


Figure 17: Rolling update: ready pods per version over time (maxSurge 1, maxUnavailable 0).

7.3 Faulty release and rollback

Metric	Value
Release 3.0 pods that became Ready	0
Release 3.0 pods stuck not-ready (maximum)	1
Rollout outcome	stalled / failed (progress deadline 60 s)
Rollback duration (rollout undo → complete)	0.2 s
Probe requests / failed	607 / 0
Availability	100.0 %
Responses by version	2.0: 607

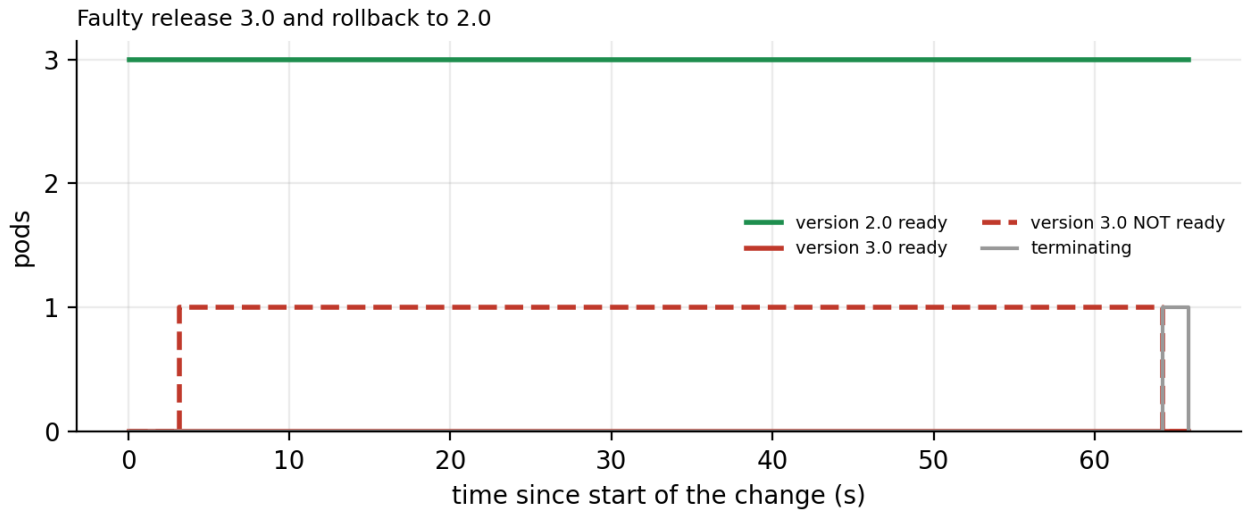


Figure 18: Faulty release 3.0 and rollback: the new pods never become ready, the old pods keep serving until the rollback.

7.4 Resource utilisation and autoscaling

Metric	Value
HPA configuration	min 3 · max 6 · target 50 % of the 100m CPU request
Load applied for	156.1 s (14 parallel clients, 100 ms of CPU work per request)
Peak CPU utilisation (% of request)	194%
Peak number of pods	6
First scale-up observed at	31.2 s
Peak replica count reached at	31.2 s
Back at minimum replicas at	200.0 s
Availability during the experiment	100.0 % (0 failed of 349)
Latency p50 / p95 / max	9.3 / 295.3 / 498.1 ms

t (s)	Phase	CPU % of request	HPA replicas (cur → desired)	Pods ready / total	Avg CPU / pod
0.0	load	10%	3 → 3	3 / 3	10m
6.2	load	10%	3 → 3	3 / 3	10m
12.4	load	10%	3 → 3	3 / 3	10m
18.7	load	32%	3 → 3	3 / 3	32m
24.9	load	32%	3 → 3	3 / 3	32m
31.2	load	194%	3 → 6	3 / 6	195m
37.4	load	194%	3 → 6	6 / 6	195m
43.6	load	194%	3 → 6	6 / 6	188m
49.9	load	193%	6 → 6	6 / 6	188m
56.1	load	193%	6 → 6	6 / 6	188m
62.4	load	188%	6 → 6	6 / 6	187m
68.6	load	188%	6 → 6	6 / 6	187m
74.8	load	188%	6 → 6	6 / 6	191m
81.1	load	190%	6 → 6	6 / 6	191m
87.3	load	190%	6 → 6	6 / 6	191m
93.5	load	185%	6 → 6	6 / 6	186m
100.0	load	185%	6 → 6	6 / 6	186m
106.2	load	190%	6 → 6	6 / 6	190m
112.5	load	190%	6 → 6	6 / 6	190m
118.7	load	190%	6 → 6	6 / 6	190m
125.0	load	189%	6 → 6	6 / 6	190m
131.2	load	189%	6 → 6	6 / 6	190m

t (s)	Phase	CPU % of request	HPA replicas (cur → desired)	Pods ready / total	Avg CPU / pod
137.4	load	188%	6 → 6	6 / 6	189m
143.7	load	188%	6 → 6	6 / 6	189m
149.9	load	188%	6 → 6	6 / 6	189m
156.1	cooldown	189%	6 → 6	6 / 6	189m
162.4	cooldown	189%	6 → 6	6 / 6	189m
168.7	cooldown	159%	6 → 6	6 / 6	160m
174.9	cooldown	159%	6 → 6	6 / 6	160m
181.2	cooldown	20%	6 → 6	6 / 6	20m
187.5	cooldown	20%	6 → 6	6 / 6	20m
193.8	cooldown	20%	6 → 6	6 / 6	15m
200.0	cooldown	14%	6 → 3	3 / 3	3m

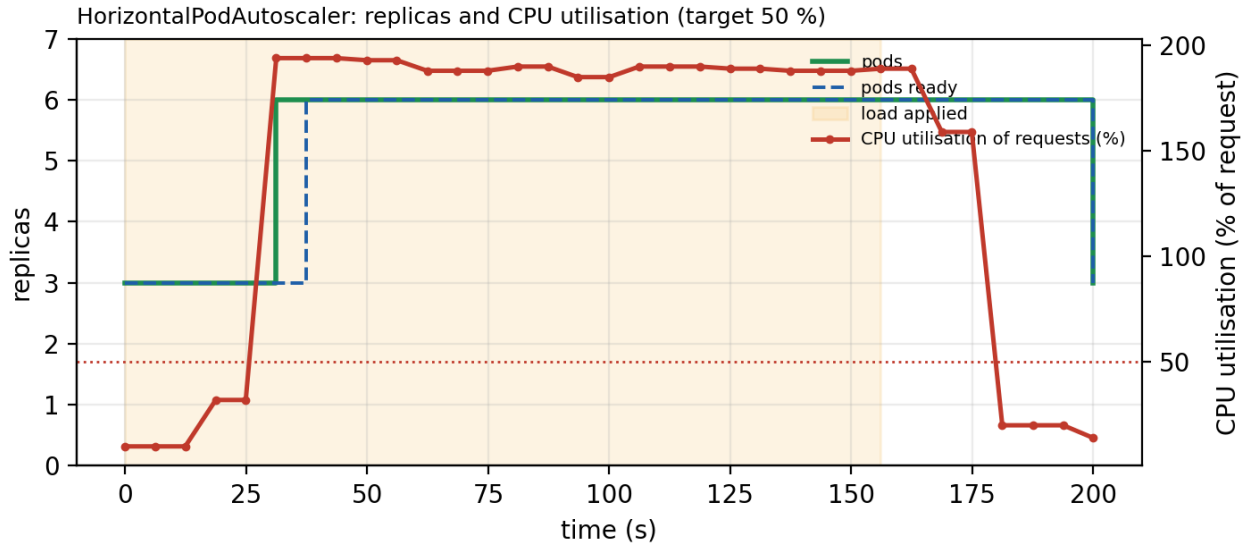


Figure 19: Autoscaling under load: replica count and CPU utilisation over time.

7.5 Evaluation criteria

Criterion	Evidence	Status
Scaling effectiveness	Manual scaling reached every target (slowest: 5.6 s); the HPA grew the Deployment to 6 pods under load and back to the minimum afterwards.	Met
Rolling update implementation	Version 2.0 replaced 1.0 pod by pod; 4 of 145 requests failed; both versions served during the transition.	Check
System stability	The faulty release never received traffic (0 failed requests), the rollout stalled instead of removing healthy pods, and rollout undo restored version 2.0; availability under load was 100.0 %.	Met

8. Observations

Scaling up is bounded by start-up, scaling down is immediate. Scaling 3 → 6 took 5.6 s (new containers must start and pass readiness); scaling 6 → 2 took 0.1 s.

maxUnavailable 0 keeps capacity constant. During the rolling update the probe saw 4 failures in 145 requests and responses from both versions (1.0: 95, 2.0: 46); the pod timeline (Section 7.2) shows how the new pods replaced the old ones.

Readiness gates protect users from a bad release. Release 3.0 never became ready, so it received no traffic; the probe still recorded 100.0 % availability while the rollout was stalled and during the rollback (0.2 s).

Rollback is a controller operation, not a rebuild. `kubectl rollout undo` re-activated the previous ReplicaSet, which Kubernetes keeps for exactly this purpose (revisionHistoryLimit 5).

The HPA scales on measured CPU, not instantly. CPU utilisation peaked at 194% of the request against a 50 % target; the first extra pod appeared at 31.2 s and 6 pods were running at the peak.

Scale-down is deliberately slower. After the load stopped the replica count returned to the minimum at 200.0 s (scale-down stabilisation window shortened to 30 s; the default is 300 s).

Requests, not limits, drive autoscaling. The utilisation percentage is relative to the 100m request, so a pod limited to 200m can show up to 200 %; choosing realistic requests is essential for sensible HPA behaviour.

Latency under load. The probe measured p50 9.3 ms and p95 295.3 ms (maximum 498.1 ms) while the pods were CPU-bound, compared with 5.8 ms p50 during the quiet rolling update.

9. Known Limitations

• **Single machine.** All nodes share one laptop's CPU, so measured CPU and latency also depend on what else the machine is doing; absolute numbers are indicative.

• **Synthetic workload.** /work burns CPU for a fixed time; real applications may be memory- or I/O-bound and need other metrics (custom or external metrics).

• **Shortened HPA cool-down.** The 30 s scale-down window is for demonstration; production setups usually keep minutes to avoid flapping.

• **Probe via NodePort and host port mapping.** Measurements include Docker Desktop's network path, not only Kubernetes.

• **No cluster autoscaling.** Only pods are scaled; the number of nodes is fixed.

10. Conclusion

Kubernetes scaled the application manually and automatically, replaced version 1.0 by 2.0 without interrupting service, contained a faulty 3.0 release through readiness probes and the rolling-update strategy, and restored the previous version with one command. Continuous probing quantified availability and latency in every experiment, and metrics-server data explained the autoscaler's decisions. The deliverables are the scaling documentation (Sections 4 and 7), the monitoring screenshots (Section 6) and the performance observations (Sections 7 and 8).